

Table of Contents

Chapters 1–4

Randomized Reward Shaping Deep Reinforcement Learning
for Self-Adaptive Systems

Alireza Kavianifar

February 15, 2026

Contents

1.1 Background and Context	4
1.1.1 Self-Adaptive Systems: motivation and examples	4
1.1.2 The MAPE-K feedback loop and the Managed/Managing System abstraction	4
1.1.3 Limitations of rule-based and static adaptation	4
1.2 Runtime Uncertainty in Self-Adaptive Systems	4
1.2.1 Sources of uncertainty (environmental, system-level)	4
1.2.2 Impact of uncertainty on adaptation decisions (examples from DeltaIoT)	4
1.3 Learning-based Adaptation: Why Reinforcement Learning?	4
1.3.1 RL benefits for runtime decision-making	4
1.3.2 Limits of classic RL in large adaptation spaces	4
1.4 What “Continuous” Means in This Thesis	4
1.4.1 Continuous = continuous runtime learning / streaming adaptation cycles (not continuous-action control)	4
1.5 Problem Statement (precise formulation extracted from the article)	4
1.6 Proposed Solution Overview: RS-DRL (high-level)	4
1.6.1 Key idea: randomized reward reshaping during experience replay	4
1.6.2 Architectural placement within MAPE-K (on-demand retraining & async learning)	4
1.7 Research Objectives and Research Questions	4
1.8 Contributions of the Thesis (expanded from the article’s contribution list)	4
1.9 Research Methodology Overview	4
1.9.1 Theoretical modeling and RL formulation	4
1.9.2 Algorithm design and reward shaping experiments	4
1.9.3 Case study (DeltaIoT): datasets, training regime, evaluation metrics	4
1.10 Thesis Organization (brief outline of subsequent chapters)	4
2.1 Self-Adaptive Systems: Concepts and Architectural Patterns	5
2.1.1 MAPE-K detailed (Monitor, Analyze, Plan, Execute, Knowledge)	5
2.1.2 Managed vs Managing System abstraction and temporal classification (monitoring vs asynchronous learning)	5
2.2 Uncertainty in Self-Adaptive Systems	5
2.2.1 Taxonomy of uncertainties (environmental, systemic, goal drift)	5
2.2.2 Effects of uncertainty on verification and adaptation (motivating evidence from DeltaIoT)	5

2.3 Reinforcement Learning Foundations for SAS	5
2.3.1 MDPs, policy/value formulation and why RL suits adaptation problems	5
2.3.2 Known challenges: local optima, generalization, retraining cost	5
2.4 Deep Reinforcement Learning in Large Discrete Adaptation Spaces	5
2.4.1 DQN architecture and rationale for discrete adaptation options (why DQN was chosen in the article)	5
2.4.2 Experience replay, target networks, stability techniques used	5
2.5 Reward Shaping and Experience Relabeling (state of the art)	5
2.5.1 HER and goal-conditioned methods — contrast to RS-DRL (no pre-defined goals)	5
2.5.2 Other generalization techniques	5
2.6 Related Work: classification and gaps	5
2.6.1 ML-based methods, search-based planners, RL-based adaptation — strengths and weaknesses (article’s survey)	5
2.6.2 Identified gap: scalable, uncertainty-aware, continuous runtime DRL for large adaptation spaces	5
2.7 Chapter Summary (lead into Chapter 3’s formal modeling)	5
3.1 Motivating Case Study: DeltaIoT System (detailed description)	6
3.1.1 Network, nodes, links, configuration parameters and examples	6
3.2 Modeling Uncertainty in DeltaIoT and SAS	6
3.2.1 Interference/noise ranges, message-load variability, data shifts (Fig.2 evidence)	6
3.2.2 How uncertainty affects QoS metrics (packet loss, latency, energy)	6
3.3 Adaptation Space and Configuration Representation	6
3.3.1 Definition: adaptation option, parameter encoding, scale (216 / 1,296 / 4,096)	6
3.3.2 Implications for search/learning complexity	6
3.4 RL Formalization for the Adaptive Problem (exact equations from paper)	6
3.4.1 State space definition (e.g., $S = (E, P, L)$)	6
3.4.2 Action space: discrete adaptation options and encoding	6
3.4.3 Reward functions: multi-objective normalization, threshold/setpoint formulations (include equations 11–12 and Table 2)	6
3.4.4 Transition dynamics (how verifier / ActivFORMS produces next states)	6
3.5 Continuous / Streaming Training Assumptions	6
3.5.1 Treating adaptation cycles as a continuous stream (training horizons and validation splits per instance)	6
3.5.2 Temporal separation: runtime adaptation vs asynchronous learning (design assumption) . . .	6
3.6 Verification & Runtime Models for Option Analysis	6
3.6.1 UPPAAL-SMC models and ActivFORMS execution engine (role in reward calculation)	6
3.7 Formal Problem Statement (thesis version) — precise objective and constraints (ready to feed Chapter 4)	6
4.1 Chapter overview & architectural perspective (scope, process-oriented organization)	7
4.2 System-level Self-Adaptation Loop (nine-stage continuous adaptation cycle)	7
4.2.1 Stage 1: Runtime data collection (sensors → Monitor)	7
4.2.2 Stage 2: State forwarding to RS-DRL module (experience collection)	7
4.2.3 Stage 3: Asynchronous model training and storage (replay → train → store)	7
4.2.4 Stage 4: State forwarding to Analyzer	7
4.2.5 Stage 5: Adaptation triggering (Analyzer detects violations)	7
4.2.6 Stage 6: Trained model retrieval	7

4.2.7 Stage 7: Adaptation option prediction (Adaptation Option Predictor)	7
4.2.8 Stage 8: Adaptation option retrieval & verification	7
4.2.9 Stage 9: Adaptation application (Executor)	7
4.3 Mapping RS-DRL to MAPE-K (conceptual mapping and roles)	7
4.4 Runtime Monitoring and State Construction	7
4.4.1 Data collection, preprocessing, feature extraction, state vector construction	7
4.5 Analysis and Adaptation Triggering	7
4.5.1 Change detection & thresholding assumptions (when retraining is triggered)	7
4.6 Planning & Adaptation Option Prediction	7
4.6.1 Adaptation Option Selector (ϵ -greedy + DQN) & architecture of predictor	7
4.7 Verification of Adaptation Options	7
4.7.1 Runtime model checking with UPPAAL-SMC & how verifier informs reward	7
4.8 Execution and Adaptation Application	7
4.8.1 Executor behavior and coordination with Knowledge repository	7
4.9 Asynchronous RS-DRL Learning and Policy Update (core algorithmic chapter)	7
4.9.1 Overall RS-DRL algorithm (Algorithm 1) and parameters ($\eta, \gamma, \epsilon, \rho$)	7
4.9.2 Replay Memory & randomized reward reshaping (Algorithm 2) — rationale & implementation	7
4.9.3 Model training, target networks, loss function, stability practices	7
4.10 Knowledge Management and Process Coordination	7
4.10.1 Knowledge repository design (models, adaptation options, state history)	7
4.10.2 Read/write coordination contracts and heterogeneous process synchronization	7
4.11 Implementation details & hyperparameter choices	7
4.11.1 Gymnasium environment, Stable-Baselines3 DQN extension, dataset traces (Gheibi & Weyns)	7
4.12 Chapter summary and transition to evaluation (Chapter 5)	7

- 1.1 Background and Context
 - 1.1.1 Self-Adaptive Systems: motivation and examples
 - 1.1.2 The MAPE-K feedback loop and the Managed/Managing System abstraction
 - 1.1.3 Limitations of rule-based and static adaptation
- 1.2 Runtime Uncertainty in Self-Adaptive Systems
 - 1.2.1 Sources of uncertainty (environmental, system-level)
 - 1.2.2 Impact of uncertainty on adaptation decisions (examples from DeltaIoT)
- 1.3 Learning-based Adaptation: Why Reinforcement Learning?
 - 1.3.1 RL benefits for runtime decision-making
 - 1.3.2 Limits of classic RL in large adaptation spaces
- 1.4 What “Continuous” Means in This Thesis
 - 1.4.1 Continuous = continuous runtime learning / streaming adaptation cycles (not continuous-action control)
- 1.5 Problem Statement (precise formulation extracted from the article)
- 1.6 Proposed Solution Overview: RS-DRL (high-level)
 - 1.6.1 Key idea: randomized reward reshaping during experience replay
 - 1.6.2 Architectural placement within MAPE-K (on-demand retraining & async learning)
- 1.7 Research Objectives and Research Questions
- 1.8 Contributions of the Thesis (expanded from the article’s contribution list)
- 1.9 Research Methodology Overview
 - 1.9.1 Theoretical modeling and RL formulation
 - 1.9.2 Algorithm design and reward shaping experiments
 - 1.9.3 Case study (DeltaIoT): datasets, training regime, evaluation metrics
- 1.10 Thesis Organization (brief outline of subsequent chapters)

- 2.1 Self-Adaptive Systems: Concepts and Architectural Patterns**
 - 2.1.1 MAPE-K detailed (Monitor, Analyze, Plan, Execute, Knowledge)
 - 2.1.2 Managed vs Managing System abstraction and temporal classification (monitoring vs asynchronous learning)
- 2.2 Uncertainty in Self-Adaptive Systems**
 - 2.2.1 Taxonomy of uncertainties (environmental, systemic, goal drift)
 - 2.2.2 Effects of uncertainty on verification and adaptation (motivating evidence from DeltaIoT)
- 2.3 Reinforcement Learning Foundations for SAS**
 - 2.3.1 MDPs, policy/value formulation and why RL suits adaptation problems
 - 2.3.2 Known challenges: local optima, generalization, retraining cost
- 2.4 Deep Reinforcement Learning in Large Discrete Adaptation Spaces**
 - 2.4.1 DQN architecture and rationale for discrete adaptation options (why DQN was chosen in the article)
 - 2.4.2 Experience replay, target networks, stability techniques used
- 2.5 Reward Shaping and Experience Relabeling (state of the art)**
 - 2.5.1 HER and goal-conditioned methods — contrast to RS-DRL (no pre-defined goals)
 - 2.5.2 Other generalization techniques
- 2.6 Related Work: classification and gaps**
 - 2.6.1 ML-based methods, search-based planners, RL-based adaptation — strengths and weaknesses (article’s survey)
 - 2.6.2 Identified gap: scalable, uncertainty-aware, continuous runtime DRL for large adaptation spaces
- 2.7 Chapter Summary (lead into Chapter 3’s formal modeling)**

- 3.1 Motivating Case Study: DeltaIoT System (detailed description)**
 - 3.1.1 Network, nodes, links, configuration parameters and examples
- 3.2 Modeling Uncertainty in DeltaIoT and SAS**
 - 3.2.1 Interference/noise ranges, message-load variability, data shifts (Fig.2 evidence)
 - 3.2.2 How uncertainty affects QoS metrics (packet loss, latency, energy)
- 3.3 Adaptation Space and Configuration Representation**
 - 3.3.1 Definition: adaptation option, parameter encoding, scale (216 / 1,296 / 4,096)
 - 3.3.2 Implications for search/learning complexity
- 3.4 RL Formalization for the Adaptive Problem (exact equations from paper)**
 - 3.4.1 State space definition (e.g., $S = (E, P, L)$)
 - 3.4.2 Action space: discrete adaptation options and encoding
 - 3.4.3 Reward functions: multi-objective normalization, threshold/setpoint formulations (include equations 11–12 and Table 2)
 - 3.4.4 Transition dynamics (how verifier / ActivFORMS produces next states)
- 3.5 Continuous / Streaming Training Assumptions**
 - 3.5.1 Treating adaptation cycles as a continuous stream (training horizons and validation splits per instance)
 - 3.5.2 Temporal separation: runtime adaptation vs asynchronous learning (design assumption)
- 3.6 Verification & Runtime Models for Option Analysis**
 - 3.6.1 UPPAAL-SMC models and ActivFORMS execution engine (role in reward calculation)
- 3.7 Formal Problem Statement (thesis version) — precise objective and constraints (ready to feed Chapter 4)**

- 4.1 Chapter overview & architectural perspective (scope, process-oriented organization)
- 4.2 System-level Self-Adaptation Loop (nine-stage continuous adaptation cycle)
 - 4.2.1 Stage 1: Runtime data collection (sensors $\rightarrow$ Monitor)
 - 4.2.2 Stage 2: State forwarding to RS-DRL module (experience collection)
 - 4.2.3 Stage 3: Asynchronous model training and storage (replay $\rightarrow$ train $\rightarrow$ store)
 - 4.2.4 Stage 4: State forwarding to Analyzer
 - 4.2.5 Stage 5: Adaptation triggering (Analyzer detects violations)
 - 4.2.6 Stage 6: Trained model retrieval
 - 4.2.7 Stage 7: Adaptation option prediction (Adaptation Option Predictor)
 - 4.2.8 Stage 8: Adaptation option retrieval & verification
 - 4.2.9 Stage 9: Adaptation application (Executor)
- 4.3 Mapping RS-DRL to MAPE-K (conceptual mapping and roles)
- 4.4 Runtime Monitoring and State Construction
 - 4.4.1 Data collection, preprocessing, feature extraction, state vector construction
- 4.5 Analysis and Adaptation Triggering
 - 4.5.1 Change detection & thresholding assumptions (when retraining is triggered)
- 4.6 Planning & Adaptation Option Prediction
 - 4.6.1 Adaptation Option Selector (ϵ -greedy + DQN) & architecture of predictor
- 4.7 Verification of Adaptation Options
 - 4.7.1 Runtime model checking with UPPAAL-SMC & how verifier informs reward
- 4.8 Execution and Adaptation Application
 - 4.8.1 Executor behavior and coordination with Knowledge repository
- 4.9 Asynchronous RS-DRL Learning and Policy Update (core algorithmic chapter)
 - 4.9.1 Overall RS-DRL algorithm (Algorithm 1) and parameters (η , γ , ϵ , ρ)
 - 4.9.2 Replay Memory & randomized reward reshaping (Algorithm 2) — rationale & implementation
 - 4.9.3 Model training, target networks, loss function, stability practices
- 4.10 Knowledge Management and Process Coordination
 - 4.10.1 Knowledge repository design (models, adaptation options, state history)
 - 4.10.2 Read/write coordination contracts and heterogeneous process synchronization
- 4.11 Implementation details & hyperparameter choices
 - 4.11.1 Gymnasium environment, Stable-Baselines3 DQN extension, dataset traces (Gheibi & Weiss)